

Problem 10: Give an algorithm for finding the height or depth of the Binary Tree.

Solution: Recursively Calculate height of left Subtree & Right Subtree of a node and assign height to the node as max of the heights of two children + 1.

This is similar to preOrder tree traversal (and DFS of graph algorithms).

Algorithm

- 1 if root == null, return 0
- 2 Recursively find height of left Subtree
- 3 Recursively find height of Right Subtree
- 4 Return 1 + max (leftHeight, rightHeight)

Code

```
public class HeightOfTree {  
  
    public static int height(TreeNode root) {  
        if (root == null)  
            return 0;  
  
        int leftHeight = height(root.left);  
        int rightHeight = height(root.right);  
  
        return 1 + Math.max(leftHeight, rightHeight);  
    }  
}
```

Problem 11: give an algorithm for finding the height or depth of the Binary Tree. Iterative Approach

Solution: can be achieved using level order traversal. This is similar to BFS of graph algorithms. End of level is identified with Null.

Algorithm

1. if root == Null, return 0
2. Create an Empty Queue Q
3. Enqueue root into Q
4. Initialize height = 0
5. While Q is not Empty
 - Set levelSize = Q.size()
 - Process all nodes of this level
 - Dequeue one node
 - Enqueue its left child (if exists)
 - Enqueue its right child (if exists)
 - Increment height by 1
6. Return Height.

Code

```
public static int height(TreeNode root) {
    if (root == null)
        return 0;

    Queue q = new LinkedList<>();
    q.add(root);

    int height = 0;

    while (!q.isEmpty()) {
        int levelSize = q.size();

        for (int i = 0; i < levelSize; i++) {
            TreeNode cur = q.poll();

            if (cur.left != null)
                q.add(cur.left);
            if (cur.right != null)
                q.add(cur.right);
        }
        height++; // completed one level
    }
    return height;
}
```

Problem 12 Give an Algorithm to find deepest node of a Binary tree

Solution: finding the Deepest node can be done using Level order Traversal

The Deepest node

The node present at the maximum depth (last level) of the tree

If Multiple Nodes exists at that level, the Rightmost one is usually considered deepest

Algorithm

1. If root == null
return null
2. Create an Empty Queue Q.
3. Enqueue root into Q
4. Initialize TreeNode deepest == null.
5. While Q is not Empty
 - Dequeue a node curr from Q
 - Assign deepest = curr
 - If curr.left exists, Enqueue it
 - If curr.right exists, Enqueue it
6. After loop ends deepest holds the deepest node
7. Return deepest.

Code

```
public static TreeNode findDeepestNode(TreeNode root) {
    if (root == null)
        return null;

    Queue<TreeNode> q = new LinkedList<>();
    q.add(root);

    TreeNode deepest = null;

    while (!q.isEmpty()) {
        deepest = q.poll();

        if (deepest.left != null)
            q.add(deepest.left);
        if (deepest.right != null)
            q.add(deepest.right);
    }
    return deepest;
}
```